K-Factor Calculator — R / Quarto Version

Guide to `R/k_factor.R` and `k-factor_calculator.qmd` · Port of `k-factor_calculator.pl` · Prepared 23 October 2026

Purpose

This is the R port of the Perl k-factor calculator. It computes a **k-factor** for each AAVSO solar observer: a personal scaling constant such that `k × (observer's Wolf number)` lines up with the reference sunspot number, **Ra**. With the default settings it gives the same results as the Perl script, and the output is a rendered HTML report instead of console text.

Files

File	Role
`R/k_factor.R`	All the logic: date conversion, database access, the k-factor calculation, and writing the output file. It has no report code, so it can be sourced and tested on its own.
`k-factor_calculator.qmd`	Quarto report. Takes the date range and options as parameters, runs the calculation, and renders a summary, a results table and a plot.
`new_k-factors.txt`	Output: tab-separated `observer` / `K-factor`, same format as the Perl version. Overwritten on each run.

Setup

R packages

`DBI`, `RMariaDB`, `dplyr`, `ggplot2`, `knitr`, `rmarkdown`, plus Quarto itself.

Database credentials

The password is no longer in the code. Add these two lines to your `~/.Renviron` file (outside the repository) and restart R:

```
SOLAR_DB_USER=your_user
SOLAR_DB_PASSWORD=your_password
```

If either is missing, the report stops with a message telling you to set them.

Running it

```
quarto render k-factor_calculator.qmd -P start_date:2024-01-01 -P end_date:2024-06-30
```

Parameters, set in the report header or with `-P name:value`:

Parameter	Default	Meaning
`start_date`	2024-01-01	Start of the period (`YYYY-MM-DD`)
`end_date`	2024-06-30	End of the period. Must be ≥ 100 days after the start and no later than the last date with an Ra value.
`min_obs`	100	Minimum observations an observer needs in the period
`zero_handling`	substitute	`"substitute"` or `"skip"` (see below)
`output_file`	new_k-factors.txt	Where the tab-separated results are written

A date range that breaks the rules stops the render with a clear error. The Perl script asked again at the prompt instead.

How the code works

1. Dates → Julian Dates

`date_to_jd()` converts a calendar date to the Julian Date at 12:00 UT, the same value the Perl script got from `mjd2jd(cal2mjd(day, month, year, 0.5))`. `jd_to_date()` goes the other way and is used in error messages.

```
date_to_jd("2013-01-03")   # 2456296
```

2. Loading data (4 queries in total)

Function	Query
`connect_solar_db()`	Opens a MariaDB/MySQL connection using the environment variables
`last_ra_jd()`	Latest JD in `sunspot_counts`, used to check the end date
`fetch_observations()`	All unrevised Wolf numbers (`obs`, `wolf`, `jd`) in the period, for every observer at once
`fetch_ra()`	Ra values (`jd`, `ra`) in the period, one per JD
`fetch_old_k()`	Each observer's current k-factor from `sun_obsconst`

The Perl script ran two queries per observer. The R version loads everything in one pass and does the per-observer work in memory. All queries use `?` placeholders, so values are never pasted into the SQL text.

3. The calculation — `compute_k_factors()`

1. Count each observer's reports and keep observers with at least `min_obs`.
2. Join each observation to the Ra value for the same JD.
3. Flag days where W or Ra is zero or missing.
4. For each observation compute `log10(Ra) − log10(W)`, handling flagged days according to `zero_handling`.
5. Per observer, k is 10 raised to the mean of those values, i.e. the **geometric mean of Ra / W**:

```
k = 10 ^ ( (1/N) × Σ [ log10(Ra_i) − log10(W_i) ] )
```

The result has one row per observer: `n_reported`, `n_used`, `n_zero_or_missing`, and `k`.

4. Zero and missing values

Setting	Behaviour
`"substitute"` (default)	Same as the Perl script: a zero or missing W or Ra counts as log10 = 0.1 (≈ 1.26). Results match past runs.
`"skip"`	Days where W or Ra is zero or missing are left out, and N counts only the days used. This is the fix suggested in the Perl script's header comment.

Observers qualify on all their reports, before any days are skipped, as in the Perl version. The `n_zero_or_missing` column shows how many days each observer was affected. In the test data, three such days moved k from 1.38 (skip) to 1.34 (substitute).

5. Report output

- **Summary:** the period, its JD range, how many observers qualified, and which zero-handling method was used.
- **Table:** observer, reports, days used, zero/missing days, new k, old k, change.
- **Plot:** new vs. old k-factor with a dashed equality line. Points above the line got a larger k-factor.
- **File:** `write_k_factors()` writes `new_k-factors.txt` with k to 3 decimal places.

Changes from the Perl version

Perl	R / Quarto
Password hard-coded in the script	Read from environment variables
Dates typed at prompts (`MM-DD-YYYY`)	Report parameters (`YYYY-MM-DD`)
Two queries per observer	Four queries in total
Zeros always replaced with 0.1	Choice of `substitute` or `skip`, with a count of affected days
Output file appended on every run	Output file overwritten
Console text only	HTML report with table and plot
Global variables, no `use strict`	Small functions with explicit inputs and outputs

Testing

The calculation was checked on made-up data against a line-by-line R copy of the Perl loop, including a zero Ra, a missing Ra and a zero Wolf number. The k-factors matched, the `skip` option dropped exactly the three flagged days, and the output file had the Perl format. The full report has not been rendered against the live AAVSO database.

Appendix — `R/k_factor.R`

```
# K-factor calculation for AAVSO solar observers.
# R port of k-factor_calculator.pl (Sara J. Beck with Matthew Templeton, 2013).

library(DBI)
library(dplyr)

# Julian Date at 12:00 UT on a calendar date (matches the Perl script's
# mjd2jd(cal2mjd(day, month, year, 0.5))).
date_to_jd <- function(date) {
  as.numeric(as.Date(date)) + 2440588
}

jd_to_date <- function(jd) {
  as.Date(floor(jd + 0.5) - 2440588)
}

connect_solar_db <- function(host = "db.aavso.org", dbname = "solar") {
  user <- Sys.getenv("SOLAR_DB_USER")
  password <- Sys.getenv("SOLAR_DB_PASSWORD")
  if (user == "" || password == "") {
    stop("Set SOLAR_DB_USER and SOLAR_DB_PASSWORD (e.g. in ~/.Renviron).", call. = FALSE)
  }
  dbConnect(RMariaDB::MariaDB(), host = host, dbname = dbname,
            user = user, password = password)
}

last_ra_jd <- function(con) {
  dbGetQuery(con, "SELECT MAX(jd) AS jd FROM sunspot_counts")$jd
}

# Every unrevised Wolf number reported in the period.
fetch_observations <- function(con, start_jd, end_jd) {
  dbGetQuery(con, "
    SELECT obs, W AS wolf, date AS jd
    FROM sun_data
    WHERE date >= ? AND date <= ? AND revised = 0",
    params = list(start_jd, end_jd))
}

# Reference sunspot numbers (Ra) for the period, one per JD.
fetch_ra <- function(con, start_jd, end_jd) {
  dbGetQuery(con, "
    SELECT jd, count AS ra
    FROM sunspot_counts
    WHERE jd >= ? AND jd <= ?",
    params = list(start_jd, end_jd)) |>
    distinct(jd, .keep_all = TRUE)
```

```
}

# Current (old) k-factor for every observer.
fetch_old_k <- function(con) {
  dbGetQuery(con, "SELECT obs, k AS old_k FROM sun_obsconst WHERE revised = 0") |>
    distinct(obs, .keep_all = TRUE)
}

# log10 that substitutes 0.1 for zero or missing values, as the Perl script does.
log10_or_substitute <- function(x) {
  ifelse(is.na(x) | x == 0, 0.1, log10(x))
}

# k = geometric mean of Ra / W over an observer's observations.
#
# zero_handling:
#   "substitute" - reproduce the Perl script: a zero or missing W or Ra
#                  contributes log10 = 0.1.
#   "skip"       - drop days where W or Ra is zero or missing.
# Observers qualify with min_obs observations in the period, counted before
# any days are skipped (as in the Perl script).
compute_k_factors <- function(observations, ra, min_obs = 100,
                              zero_handling = c("substitute", "skip")) {
  zero_handling <- match.arg(zero_handling)

  obs <- observations |>
    add_count(obs, name = "n_reported") |>
    filter(n_reported >= min_obs) |>
    left_join(ra, by = "jd") |>
    mutate(zero_or_missing = is.na(wolf) | wolf == 0 | is.na(ra) | ra == 0)

  if (zero_handling == "skip") {
    obs <- obs |>
      filter(!zero_or_missing) |>
      mutate(log_ratio = log10(ra) - log10(wolf))
  } else {
    obs <- obs |>
      mutate(log_ratio = log10_or_substitute(ra) - log10_or_substitute(wolf))
  }

  obs |>
    group_by(obs) |>
    summarise(
      n_reported = first(n_reported),
      n_used = n(),
      n_zero_or_missing = sum(zero_or_missing),
      k = 10^mean(log_ratio),
      .groups = "drop"
    ) |>
    arrange(obs)
}
```

```
write_k_factors <- function(k_factors, path = "new_k-factors.txt") {
  out <- data.frame(observer = k_factors$obs,
                    `K-factor` = sprintf("%.3f", k_factors$k),
                    check.names = FALSE)
  write.table(out, path, sep = "\t", quote = FALSE, row.names = FALSE)
  invisible(path)
}
```

Appendix — `k-factor_calculator.qmd`

````
---
title: "K-Factor Calculator"
subtitle: "AAVSO solar observers"
date: today
format:
  html:
    embed-resources: true
    toc: true
execute:
  echo: false
  warning: false
params:
  start_date: "2024-01-01"
  end_date: "2024-06-30"
  min_obs: 100
  zero_handling: "substitute"   # "substitute" (as in the Perl script) or "skip"
  output_file: "new_k-factors.txt"
---

```{r}
#| label: setup
library(dplyr)
library(ggplot2)
library(knitr)
source("R/k_factor.R")

start_jd <- date_to_jd(params$start_date)
end_jd <- date_to_jd(params$end_date)
```

```{r}
#| label: load-data
con <- connect_solar_db()

last_jd <- last_ra_jd(con)
if (end_jd > last_jd) {
  stop("End date must be no later than ", jd_to_date(last_jd),
       " (the last date with an Ra value).", call. = FALSE)
````
````

````
}
if (end_jd - start_jd < 100) {
  stop("End date must be at least 100 days after the start date.", call. = FALSE)
}

observations <- fetch_observations(con, start_jd, end_jd)
ra <- fetch_ra(con, start_jd, end_jd)
old_k <- fetch_old_k(con)
dbDisconnect(con)
```

```{r}
#| label: compute
k_factors <- compute_k_factors(observations, ra,
                               min_obs = params$min_obs,
                               zero_handling = params$zero_handling) |>
  left_join(old_k, by = "obs") |>
  mutate(change = k - old_k)

write_k_factors(k_factors, params$output_file)
```

## Summary

Period **`r params$start_date`** to **`r params$end_date`**
(JD `r start_jd` to `r end_jd`).
**`r nrow(k_factors)`** observers had at least `r params$min_obs` observations.
Zero or missing values were handled with the **`r params$zero_handling`** method.
Results were written to `` `r params$output_file` ``.

Each k-factor is the geometric mean of Ra / W over the observer's
observations:

$$
k = 10^{\frac{1}{N}\sum_{i=1}^{N} \left(\log_{10} Ra_i - \log_{10} W_i\right)}
$$

## K-factors

```{r}
#| label: tbl-k-factors
k_factors |>
  select(Observer = obs, Reported = n_reported, Used = n_used,
         `Zero/missing` = n_zero_or_missing,
         `New k` = k, `Old k` = old_k, Change = change) |>
  kable(digits = 3)
```

## New vs. old k-factors

```{r}
````

```
#| label: fig-new-vs-old
#| fig-cap: "Points above the dashed line got a larger k-factor than before."
k_factors |>
  filter(!is.na(old_k)) |>
  ggplot(aes(old_k, k)) +
  geom_abline(linetype = "dashed", colour = "grey50") +
  geom_point(colour = "#12355b", size = 2) +
  geom_text(aes(label = obs), vjust = -0.8, size = 3) +
  coord_equal() +
  labs(x = "Old k-factor", y = "New k-factor") +
  theme_minimal()
```